

Multimodal visual understand+SLAM navigation + visual line patrolling(Text Version)

Multimodal visual understand+SLAM navigation + visual line patrolling(Text Version)

1. Course Content
2. Preparation
 - 2.1 Content Description
3. Run Case
 - 3.1 Starting the Program
 - 3.1.1 Jetson Nano board Startup Steps:
 - 3.1.1 RDKX5, Raspberry Pi PI5 and ORIN board startup steps:
 - 3.2 Test Case
 - 3.2.1 Case 1
4. Source Code Analysis

1. Course Content

1. Learn to use the robot's visual understanding, line following, SLAM navigation, and other complex functional cases.
2. Study the key source code newly introduced in the tutorial.
3. **Note: This section requires you to complete the map file configuration as described in the previous section [7. Multimodal Visual Understand + SLAM Navigation].**

2. Preparation

2.1 Content Description

This section uses the Raspberry Pi 5 as an example. For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this section in the terminal. For instructions on entering the Docker container from the host computer, refer to **[01. Robot Configuration and Operation Guide] -- [5.Enter Docker (For JETSON Nano and RPi 5)]**. For RDKX5 and Orin boards, simply open a terminal and enter the commands mentioned in this course.

 This example uses `model:"qwen/qwen2.5-v1-72b-instruct:free", "qwen-v1-latest"`

 The responses from the big model for the same test command may not be exactly the same and may differ slightly from the screenshots in the tutorial. To increase or decrease the diversity of the big model's responses, refer to the section on configuring the decision-making big model parameters in **[04.AI Large Model Development] -- [5.Deploy the RAG knowledge base]** .

3. Run Case

3.1 Starting the Program

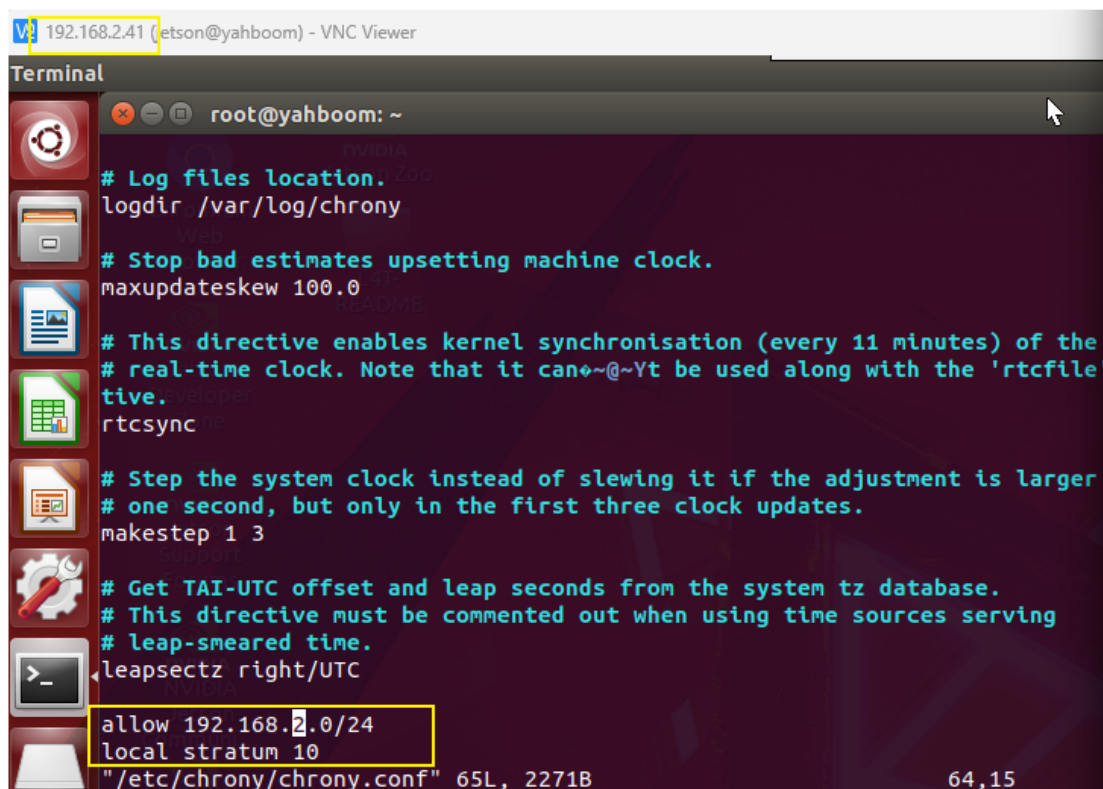
3.1.1 Jetson Nano board Startup Steps:

Due to Nano performance issues, navigation-related nodes must be run on a virtual machine. Therefore, navigation performance is highly dependent on the network. We recommend running in an indoor environment with a good network connection. You must first configure the following:

- **Board Side (Requires Docker)**

Open a terminal and enter

```
sudo vi /etc/chrony/chrony.conf
```



```
192.168.2.41 (jetson@yahboom) - VNC Viewer
Terminal
root@yahboom: ~
# Log files location.
logdir /var/log/chrony

# Stop bad estimates upsetting machine clock.
maxupdateskew 100.0

# This directive enables kernel synchronisation (every 11 minutes) of the
# real-time clock. Note that it can be used along with the 'rtcfile'
# directive.
rtcsync

# Step the system clock instead of slewing it if the adjustment is larger
# one second, but only in the first three clock updates.
makestep 1 3

# Get TAI-UTC offset and leap seconds from the system tz database.
# This directive must be commented out when using time sources serving
# leap-smear time.
# leapsectz right/UTC

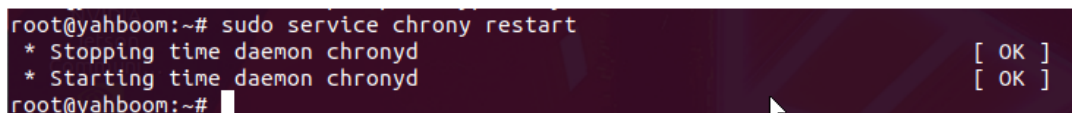
allow 192.168.2.0/24
local stratum 10
"/etc/chrony/chrony.conf" 65L, 2271B 64,15
```

Add the following two lines to the end of the file: (Fill in 192.168.2.0/24 based on the actual network segment; this example uses the current board IP address of 192.168.2.41.)

```
allow 192.168.x.0/24 #x indicates the corresponding network segment
local stratum 10
```

After saving and exiting, enter the following command to take effect:

```
sudo service chrony restart
```



```
root@yahboom:~# sudo service chrony restart
* Stopping time daemon chronyd [ OK ]
* Starting time daemon chronyd [ OK ]
root@yahboom:~#
```

- **Virtual Machine**

Open a terminal and enter

```
echo "server 192.168.2.41 iburst" | sudo tee
/etc/chrony/sources.d/jetson.sources
```

The 192.168.2.41 entry above is the board's IP address.

Enter the following command to take effect.

```
sudo chronyc reload sources
```

Enter the following command again to check the latency. If the IP address appears, it's normal.

```
chronyc sources -v
```

```
yahboom@VM:~$ chronyc sources -v

.-- Source mode  '^' = server, '=' = peer, '#' = local clock.
/ .- Source state '*' = current best, '+' = combined, '-' = not combined,
| /      'x' = may be in error, '~' = too variable, '?' = unusable.
||
||      Reachability register (octal) --.      |      .- xxxx [ yyyy ] +/- zzzz
||      Log2(Polling interval) --.      |      |      xxxx = adjusted offset,
||                                  |      |      yyyy = measured offset,
||                                  |      |      zzzz = estimated error.
||                                  |      |
||                                  |      |
MS Name/IP address             Stratum Poll Reach LastRx Last sample
=====
^- prod-ntp-5.ntp1.ps5.cano>    2  7   277   133  -4048us[-5395us] +/- 132ms
^- alphyn.canonical.com        2  8   177    66  -193us[-193us]  +/- 134ms
^- prod-ntp-3.ntp1.ps5.cano>    2  8   367   131   -16ms[-18ms]  +/- 127ms
^- prod-ntp-4.ntp4.ps5.cano>    2  8   377   129 -4907us[-6254us] +/- 133ms
^* time.nju.edu.cn             1  7   377    69   +77us[-1270us] +/- 17ms
^+ 139.199.215.251             2  8   377   135 +2358us[+1011us] +/- 46ms
^+ 119.28.183.184              2  7   377   193 +2061us[+713us]  +/- 28ms
^+ 119.28.206.193              2  8   367    8  +2058us[+2058us] +/- 36ms
^+ 192.168.2.41                4  6   377    6   -12ms[-12ms]  +/- 92ms
yahboom@VM:~$
```

- **Start the Program**

Open the host machine terminal and start the Dify environment:

```
sh ~/bringup_dify.sh
```

Open the terminal in Docker and enter the following command:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After initialization, the following content will be displayed:

```
[usb_cam_node_exe-1] YUYV 4:2:2 800 x 600 (20 Hz)
[usb_cam_node_exe-1] YUYV 4:2:2 1280 x 1024 (6 Hz)
[usb_cam_node_exe-1] YUYV 4:2:2 320 x 240 (30 Hz)
[usb_cam_node_exe-1] [INFO] [1765965894.552487562] [usb_cam]: Setting 'white_balance_temperature_auto' to 1
[usb_cam_node_exe-1] [INFO] [1765965894.552563191] [usb_cam]: Setting 'exposure_auto' to 1
[usb_cam_node_exe-1] [INFO] [1765965894.552578635] [usb_cam]: Setting 'exposure' to 150
[usb_cam_node_exe-1] [INFO] [1765965894.585467996] [usb_cam]: Setting 'focus_auto' to 0
[usb_cam_node_exe-1] [INFO] [1765965894.673818636] [usb_cam]: Timer triggering every 8 ms
[INFO] [model_service-12]: process started with pid [439]
[INFO] [action_service_usb-13]: process started with pid [441]
[action_service_usb-13] [INFO] [1765965908.031574888] [action_service_usb]: ROS_Action_Service Initialization completed
[model_service-12] [INFO] [1765965975.153285944] [model_service]: Dify LLM connection successful
[model_service-12] [INFO] [1765965975.154461154] [model_service]: ROS_LLM_Service Initialization completed
```

On the virtual machine, create a new terminal and start.

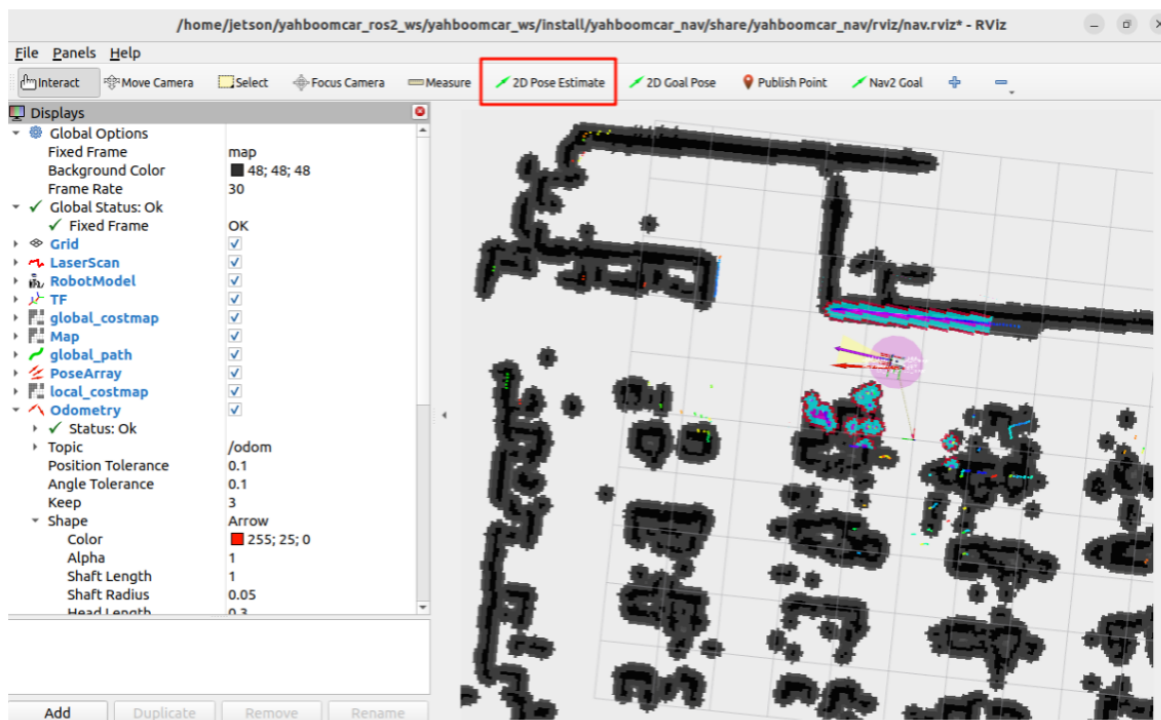
```
ros2 launch yahboomcar_nav display_nav_launch.py
```

Wait for the navigation algorithm to start before displaying the map. Then open a VM terminal and enter:

```
ros2 launch yahboomcar_nav navigation_teb_launch.py
```

Note: When running the car's mapping function, you must enter the save map command on the VM to save the navigation map.

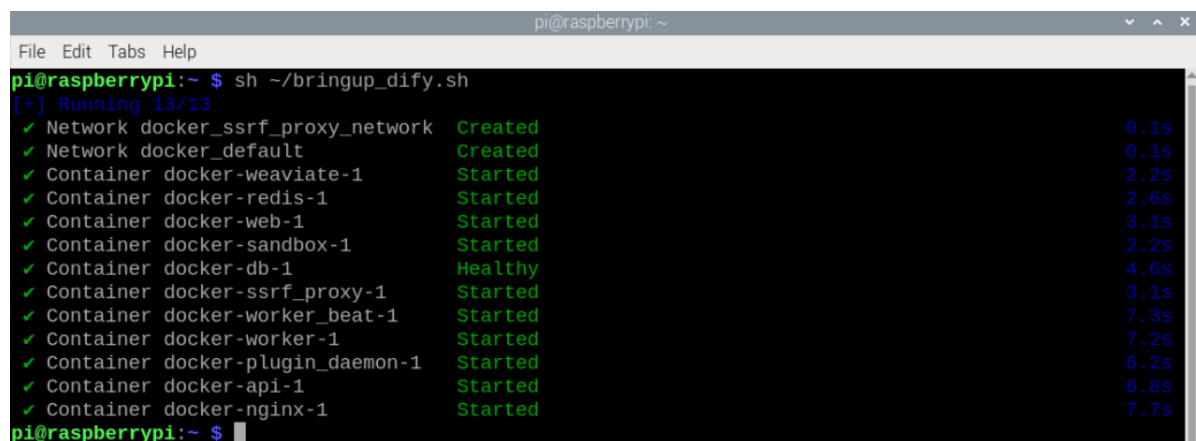
After that, follow the navigation function startup process to initialize positioning. The rviz2 visualization interface will open. Click **2D Pose Estimate** in the upper toolbar to select it. Roughly mark the robot's position and orientation on the map. After initializing positioning, preparations are complete.



3.1.1 RDKX5, Raspberry Pi PI5 and ORIN board startup steps:

Open the host terminal and start the Dify environment.

```
sh ~/bringup_dify.sh
```



For Raspberry Pi 5, you need to first enter the Docker container. For RDKX5 and Orin board, this is not necessary.

Open a terminal in Docker and enter the following command:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```

After initialization is complete, the following will be displayed:

```
[usb_cam_node_exe-1] YUYV 4:2:2 800 x 600 (20 Hz)
[usb_cam_node_exe-1] YUYV 4:2:2 1280 x 1024 (6 Hz)
[usb_cam_node_exe-1] YUYV 4:2:2 320 x 240 (30 Hz)
[usb_cam_node_exe-1] [INFO] [1765965894.552487562] [usb_cam]: Setting 'white_balance_temperature_auto' to 1
[usb_cam_node_exe-1] [INFO] [1765965894.552563191] [usb_cam]: Setting 'exposure_auto' to 1
[usb_cam_node_exe-1] [INFO] [1765965894.552578635] [usb_cam]: Setting 'exposure' to 150
[usb_cam_node_exe-1] [INFO] [1765965894.585467996] [usb_cam]: Setting 'focus_auto' to 0
[usb_cam_node_exe-1] [INFO] [1765965894.673818636] [usb_cam]: Timer triggering every 8 ms
[INFO] [model_service-12]: process started with pid [439]
[INFO] [action_service_usb-13]: process started with pid [441]
[action_service_usb-13] [INFO] [1765965908.031574888] [action_service_usb]: ROS_Action_Service Initialization completed
[model_service-12] [INFO] [1765965975.153285944] [model_service]: Dify LLM connection successful
[model_service-12] [INFO] [1765965975.154461154] [model_service]: ROS_LLM_Service Initialization completed
```

Create a new terminal on the virtual machine and start the command. **(For RDKX5, Raspberry Pi and Jetson Nano, it is recommended to run visualization in the virtual machine)**

```
ros2 launch yahboomcar_nav display_nav_launch.py
```

Wait for the navigation algorithm to start and the image will be displayed. Then open the Docker terminal and enter the following command:

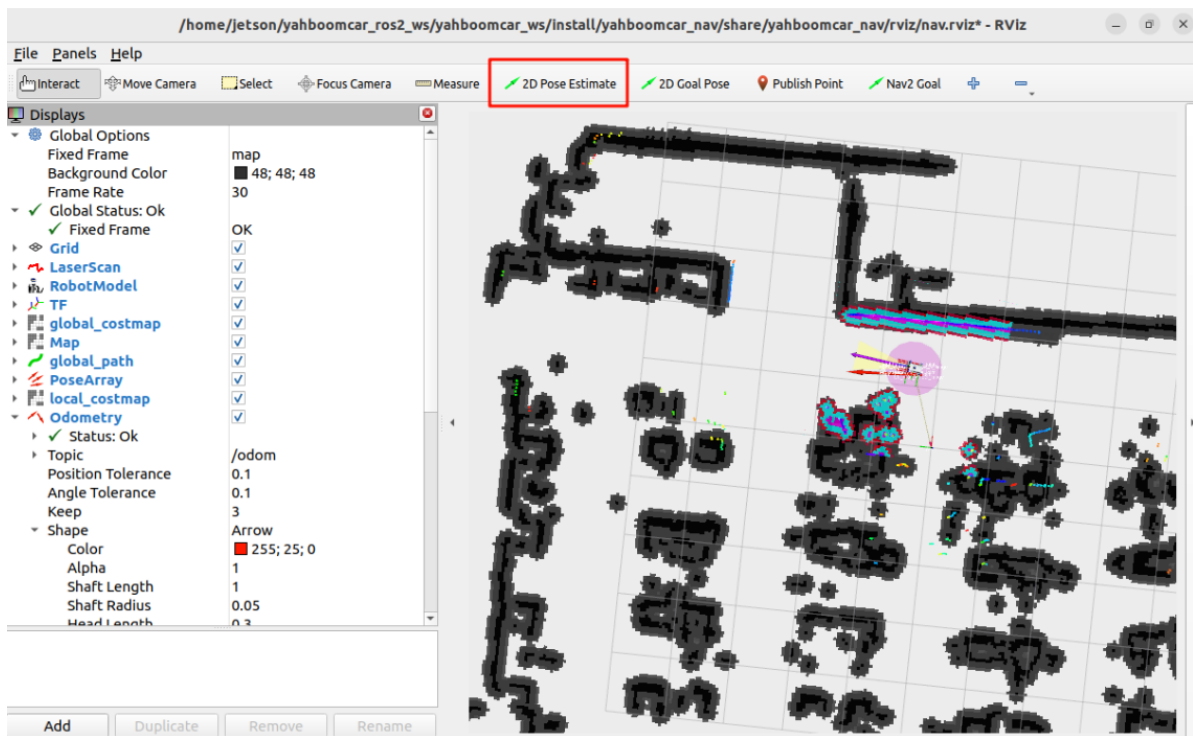
```
#Choose one of the two navigation algorithms
#Normal navigation
ros2 launch yahboomcar_nav navigation_teb_launch.py

#Fast Relocation Navigation (Not supported on RDKX5, Raspberry Pi 5, and Jetson Nano main controllers)
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=true
load_state_filename:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.pbstream

ros2 launch yahboomcar_nav navigation_cartodwb_launch.py
maps:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.yaml
```

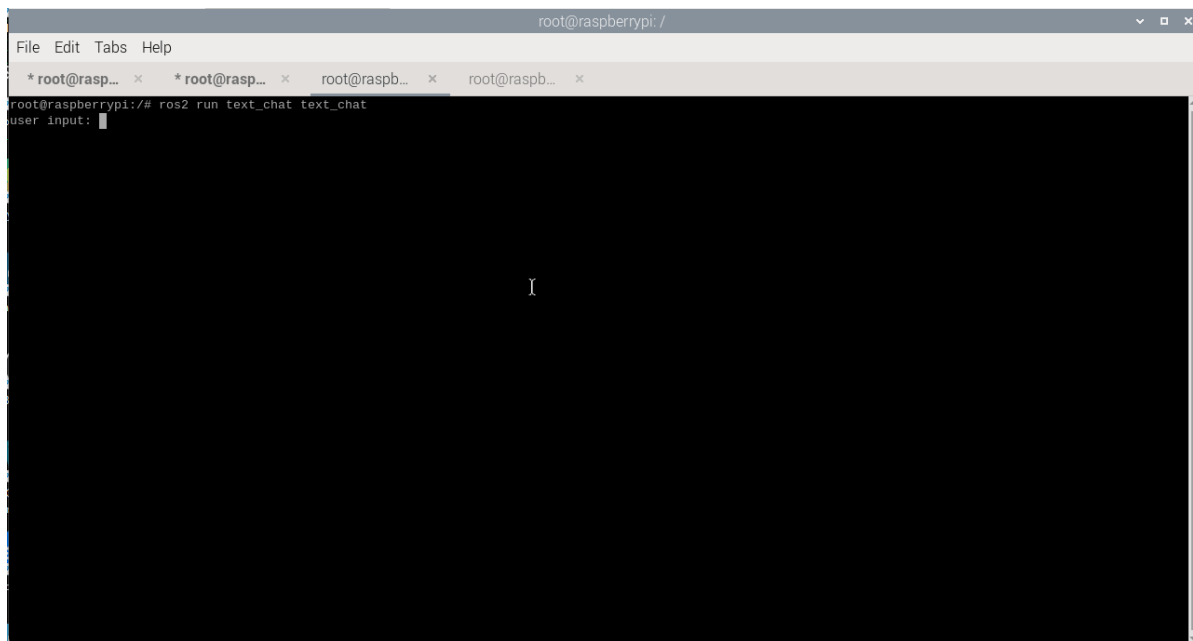
Note: yahboomcar.yaml and yahboomcar.pbstream must be mapped simultaneously, meaning they are the same map. Refer to the cartograph mapping algorithm for saving maps.

After that, follow the navigation function startup process to initialize positioning. The rviz2 visualization interface will open. Click **2D Pose Estimate** in the upper toolbar to enter the selection state. Roughly mark the robot's position and orientation on the map. After initial positioning, preparations are complete.



Open the same docker in multiple terminals and start it.

```
ros2 run text_chat text_chat
```



3.2 Test Case

Here are some reference test cases; users can create their own dialogue commands.

- Move forward 0.5 meters and back 0.5 meters, record the current location, navigate to the tea room, and observe if there's a green line on the floor. If there is, automatically patrol the green line. After patrolling, return to the starting point.

3.2.1 Case 1

In the terminal, enter "Move forward 0.5 meters and back 0.5 meters, record the current location, navigate to the tea room, and observe if there's a green line on the floor. If there is, automatically patrol the green line. After patrolling, return to the starting point." The terminal prints the following information.

```
root@raspberrypi: /
File Edit Tabs Help
*root@raspb... x *root@raspb... x root@raspb... x root@raspb... x
root@raspberrypi: /# ros2 run text_chat text_chat
user input: 前进0.5米后退0.5米，记录当前位置，导航到茶水间观察地面有没有绿线，有绿线的话进行自动巡线功能，巡线结束之后返回出发点
okay~, let me think for a moment... \[INFO] [1755687884.715571559] [text_chat_node]: 决策层AI规划:1. 前进0.5米后退0.5米
2. 调用记录当前位置函数，记录位置
3. 导航到茶水间
4. 调用视觉函数观察地面是否有绿线
5. 如果地面上有绿线，调用循迹自动驾驶函数，参数为'green'；如果没有绿线，则不执行巡线动作
6. 调用返回初始位置函数回到出发时的位置
user input: \[INFO] [1755687888.305648657] [text_chat_node]: "action": ['set_cmdvel(0.5, 0, 0, 1)', 'set_cmdvel(-0.5, 0, 0, 1)'], "response": "好呀，我先前进0.5米，再后退0.5米，做个小舞者一样热身~"
\[INFO] [1755687891.516284395] [text_chat_node]: "action": ['get_current_pose()'], "response": "现在我记下当前位置，这样等会儿就能准确地回到起点啦"
\[INFO] [1755687892.984184714] [text_chat_node]: "action": ['navigation(A)'], "response": "出发咯，我正朝着茶水间前进，做个小探险家一样"
\[INFO] [1755687936.109627075] [text_chat_node]: "action": ['seewhat()'], "response": "我已经到达茶水间了，现在用我的火眼金睛看看地面上有没有绿色的线"
\[INFO] [1755687938.635644577] [text_chat_node]: "action": ['follow_line(green)'], "response": "我看到地面上有绿色的线啦，现在开始自动巡线，就像在跳舞一样"
\[INFO] [1755687959.766541005] [text_chat_node]: "action": ['navigation(zero)'], "response": "巡线任务完成啦，现在我正返回出发时的位置，回家的路总是最安心的"
\[INFO] [1755688032.985016725] [text_chat_node]: "action": ['finishtask()'], "response": "我已经完成全部任务啦，从前进后退到巡线返回，一套动作行云流水，有需要再叫我哦~"
```

The decision-making layer model outputs the planned task steps:

```
okay~, let me think for a moment... \[INFO] [1755687884.715571559] [text_chat_node]: 决策层AI规划:1. 前进0.5米后退0.5米
2. 调用记录当前位置函数，记录位置
3. 导航到茶水间
4. 调用视觉函数观察地面是否有绿线
5. 如果地面上有绿线，调用循迹自动驾驶函数，参数为'green'；如果没有绿线，则不执行巡线动作
6. 调用返回初始位置函数回到出发时的位置
```

The execution layer model will then execute the task steps:

```
\[INFO] [1755687891.516284395] [text_chat_node]: "action": ['get_current_pose()'], "response": "现在我记下当前位置，这样等会儿就能准确地回到起点啦"
\[INFO] [1755687892.984184714] [text_chat_node]: "action": ['navigation(A)'], "response": "出发咯，我正朝着茶水间前进，做个小探险家一样"
\[INFO] [1755687936.109627075] [text_chat_node]: "action": ['seewhat()'], "response": "我已经到达茶水间了，现在用我的火眼金睛看看地面上有没有绿色的线"
\[INFO] [1755687938.635644577] [text_chat_node]: "action": ['follow_line(green)'], "response": "我看到地面上有绿色的线啦，现在开始自动巡线，就像在跳舞一样"
\[INFO] [1755687959.766541005] [text_chat_node]: "action": ['navigation(zero)'], "response": "巡线任务完成啦，现在我正返回出发时的位置，回家的路总是最安心的"
\[INFO] [1755688032.985016725] [text_chat_node]: "action": ['finishtask()'], "response": "我已经完成全部任务啦，从前进后退到巡线返回，一套动作行云流水，有需要再叫我哦~"
```

After completing the task, the robot will enter a waiting state. During this time, commands are directly passed to the execution layer model, and all conversation history is retained. You can press the **ENTER** key in the terminal to continue the conversation and enter the "End current task" command to terminate the current task cycle and start a new one.


```
root@raspberrypi: /
File Edit Tabs Help
* root@raspb... * root@raspb... * root@raspb... * root@raspb...
root@raspberrypi: /# ros2 run text_chat text_chat
user input: 前进0.5米后退0.5米,记录当前位置,导航到茶水间观察地面有没有绿线,有绿线的话进行自动巡绿线功能,巡线结束之后返回出发点
okay☺, let me think for a moment... \[INFO] [1755687884.715571559] [text_chat_node]: 决策层AI规划:1. 前进0.5米后退0.5米
2. 调用记录当前位置函数,记录位置
3. 导航到茶水间
4. 调用视觉函数观察地面是否有绿线
5. 如果地面上有绿线,调用循迹自动驾驶函数,参数为'green';如果没有绿线,则不执行巡线动作
6. 调用返回初始位置函数回到出发时的位置
user input: \[INFO] [1755687888.305648657] [text_chat_node]: "action": ['set_cmdvel(0.5, 0, 0, 1)', 'set_cmdvel(-0.5, 0, 0, 1)'], "response": 好呀,我先进0.5米,再后退0.5米,像个小舞者一样热身~
\[INFO] [1755687891.516284395] [text_chat_node]: "action": ['get_current_pose()'], "response": 现在我记下当前位置,这样等会儿就能准确地回到起点啦
\[INFO] [1755687892.984184714] [text_chat_node]: "action": ['navigation(A)'], "response": 出发咯,我正朝着茶水间前进,像个小探险家一样
\[INFO] [1755687936.109627075] [text_chat_node]: "action": ['seewhat()'], "response": 我已经到达茶水间了,现在用我的火眼金睛看看地面上有没有绿色的线
\[INFO] [1755687938.635644577] [text_chat_node]: "action": ['follow_line(green)'], "response": 我看到地面上有绿色的线啦,现在开始自动巡绿线,就像在跳舞一样
\[INFO] [1755687959.766541005] [text_chat_node]: "action": ['navigation(zero)'], "response": 巡线任务完成啦,现在我正返回出发时的位置,回家的路总是最安心的
\[INFO] [1755688032.985016725] [text_chat_node]: "action": ['finishtask()'], "response": 我已经完成全部任务啦,从前进后退到巡线返回,一套动作行云流水,有需要再叫我哦~
user input: 结束当前任务
okay☺, let me think for a moment... \[INFO] [1755688265.628314298] [text_chat_node]: "action": ['finish_dialogue()'], "response": 好的,任务已经结束了,有需要再叫我哦~
user input: █
```

4. Source Code Analysis

Source code location:

Jetson Orin Nano:

```
#NUWA camera user
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/largemodel/action_service_nuwa.py
#USB camera user
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/largemodel/action_service_usb.py
```

Jetson Nano, Raspberry Pi:

You need to first enter Docker.

```
#NUWA Camera User
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/largemodel/action_service_nuwa.py
#USB Camera User
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/largemodel/action_service_usb.py
```

RDKX5:

```
#NUWA Camera User
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
#USB Camera User
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

This example uses the **seewhat**, **navigation**, **load_target_points**, **get_current_pose**, and **follow_line(self, color)** methods from the **CustomActionServer** class. Most of these methods have been explained in the **Multimodal Visual Understanding + SLAM Navigation** section. For a detailed explanation of the **follow_line(self, color)** function, refer to the **Multimodal Visual**

Understanding + Visual Line Patrolling section.